Session 6: Grouping and SubqueriesTrainer Coverage:GROUP BY, HAVING.Subqueries and nested queries.Example: “Show users whose average daily spend > ?500.”Pending

Tasks

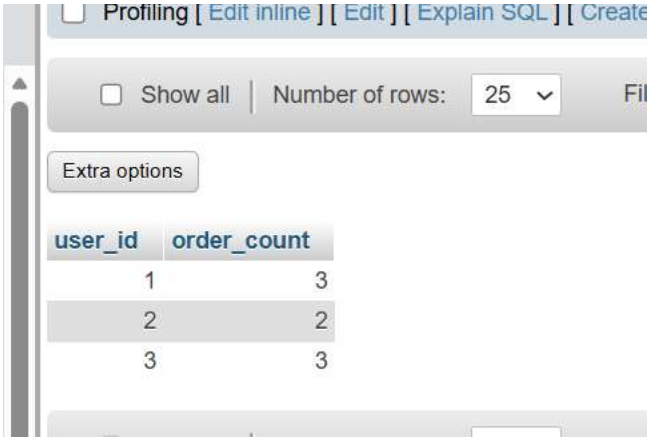
1) **Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.**

QUERY:

```sql
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    order_date DATE
);

INSERT INTO orders (user_id, order_date) VALUES
(1, '2026-08-01'),
(1, '2026-08-05'),
(1, '2026-08-10'),
(2, '2026-08-02'),
(2, '2026-08-07'),
(3, '2026-08-03'),
(3, '2026-08-08'),
(3, '2026-08-15');
```

```sql

SELECT
    user_id,
    COUNT(*) AS order_count
FROM orders
GROUP BY user_id;
```

OUTPUT:


Profiling [Edit inline] [Edit] [Explain SQL] [Create
Show all
Number of rows:
25
Extra options
user_id
order_count
1
3
2
2
3
3

2) **Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.**

QUERY:

Run SQL query/queries on table my database.restaurant:

```
1 SELECT name, AVG(rating) AS average_rating
2 FROM restaurant
3 GROUP BY name
4 HAVING AVG(rating) > 4.0;
```

OUTPUT:

Extra options

	name	average_rating
Edit Copy Delete	Dragon Wok	4.600000
Edit Copy Delete	Spice Garden	4.700000
Edit Copy Delete	Sushi World	4.700000

3) **Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.**

QUERY:

Run SQL query/queries on database my database:

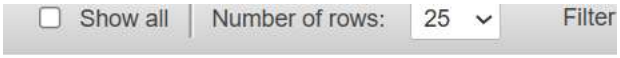
```sql
CREATE TABLE payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_date DATE
);

INSERT INTO payments (user_id, amount, payment_date) VALUES
(1, 1500.00, '2026-08-01'),
(1, 2500.00, '2026-08-05'),
(2, 1800.00, '2026-08-03'),
(2, 3000.00, '2026-08-07'),
(3, 1200.00, '2026-08-04'),
(3, 1900.00, '2026-08-08'),
(4, 5000.00, '2026-08-10');
```

```sql
SELECT DISTINCT user_id
FROM payments
WHERE amount > 2000;
```

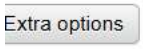
OUTPUT:

Extra options

	payment_id	user_id	amount	payment_date
Edit Copy Delete	1	1	1500.00	2026-08-01
Edit Copy Delete	2	1	2500.00	2026-08-05
Edit Copy Delete	3	2	1800.00	2026-08-03
Edit Copy Delete	4	2	3000.00	2026-08-07
Edit Copy Delete	5	3	1200.00	2026-08-04
Edit Copy Delete	6	3	1900.00	2026-08-08
Edit Copy Delete	7	4	5000.00	2026-08-10



Show all | Number of rows: 25 Filter



Extra options

	user_id
Edit Copy Delete	1
Edit Copy Delete	2
Edit Copy Delete	4

4) **Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.

Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.**

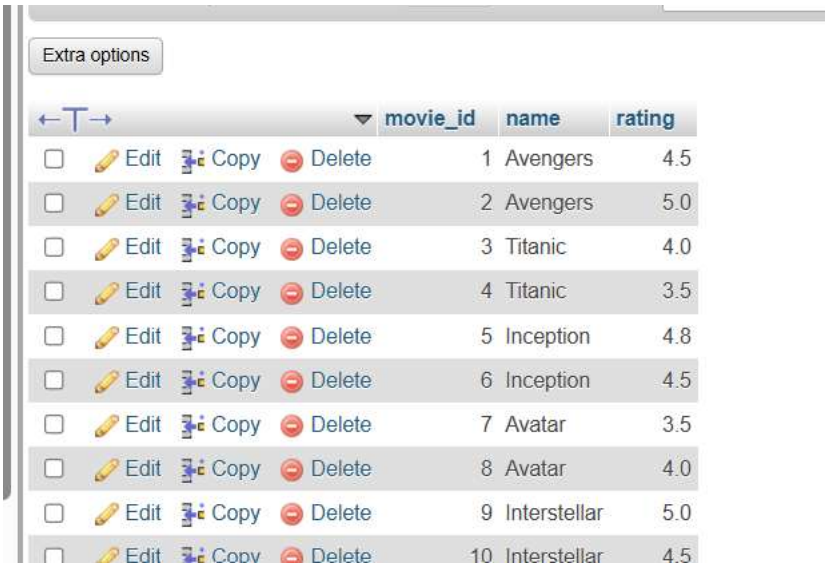
QUERY:

```sql
CREATE TABLE movies (
    movie_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    rating DECIMAL(3,1) NOT NULL
);

INSERT INTO movies (name, rating) VALUES
('Avengers', 4.5),
('Avengers', 5.0),
('Titanic', 4.0),
('Titanic', 3.5),
('Inception', 4.8),
('Inception', 4.5),
('Avatar', 3.5),
('Avatar', 4.0),
```

```sql
SELECT name, AVG(rating) AS average_rating
FROM movies
GROUP BY name
HAVING AVG(rating) > (
    SELECT AVG(rating)
    FROM movies
);
```

```sql
SELECT AVG(rating)
FROM movies;
```

OUTPUT:



Extra options

←T→	movie_id	name	rating
Edit Copy Delete	1	Avengers	4.5
Edit Copy Delete	2	Avengers	5.0
Edit Copy Delete	3	Titanic	4.0
Edit Copy Delete	4	Titanic	3.5
Edit Copy Delete	5	Inception	4.8
Edit Copy Delete	6	Inception	4.5
Edit Copy Delete	7	Avatar	3.5
Edit Copy Delete	8	Avatar	4.0
Edit Copy Delete	9	Interstellar	5.0
Edit Copy Delete	10	Interstellar	4.5

Show all | Number of rows: 25 Filter rows: Search

Extra options

←T→	name	average_rating
Edit Copy Delete	Avengers	4.75000
Edit Copy Delete	Inception	4.65000
Edit Copy Delete	Interstellar	4.75000

Extra options

AVG(rating)
4.33000